

Foundations of Data Science for Biologists

Favorite Python packages II

BIO 724D

2026-MAR-25

Instructors: Greg Wray and Kayla Wilhoit

Learning objectives

Understand basic concepts of object-oriented programming

Learn about the ETE4 library and how it implements OOP concepts

Learn about some other useful Python libraries

Object-oriented programming

Why OOP simplifies your life as a programmer

Consider the following code:

```
my_sum = 40 + 2  
my_sentence = "Hello" + "world"  
my_vector = vector_1 + vector_2
```

In each case, Python interprets the **+** operator in a different way

We get the same results, even if we "hide" the data type using a variable name

```
a, b, c, d = 40, 2, "Hello", "world"  
my_sum = a + b  
my_sentence = c + d
```

This behavior is intuitive and simplifies our lives; but how is it possible?

"Everything is an object in Python"

What does that mean?

For data objects

Data is "packaged" with metadata when it is stored

An essential piece of that metadata is the **class** it belongs to

The class defines how to apply operators and functions to specific data objects

Classes

A **class** is a definition or blueprint for creating a data object:

- What data is stored (from simple values to complex data structures)

- What metadata are associated (e.g., class, length, dimensions, etc.)

- How to apply operators (e.g., `+`) and functions (e.g., `len()`)

- Optionally, additional data that is shared by objects of that class

- Optionally, functions called methods that only work with objects of that class

Instantiation: creating a data object based on a class

- This is done by assigning values to a variable name

- Built-in classes: class is inferred, constructor function is called behind the scenes

- Custom classes: the constructor function must be called explicitly

Key concepts of OOP

Inheritance: we can use a class to define a new class

The new class inherits the functionality of the original class, but can extend it

The new class can also over-ride and modify functionality (**polymorphism**)

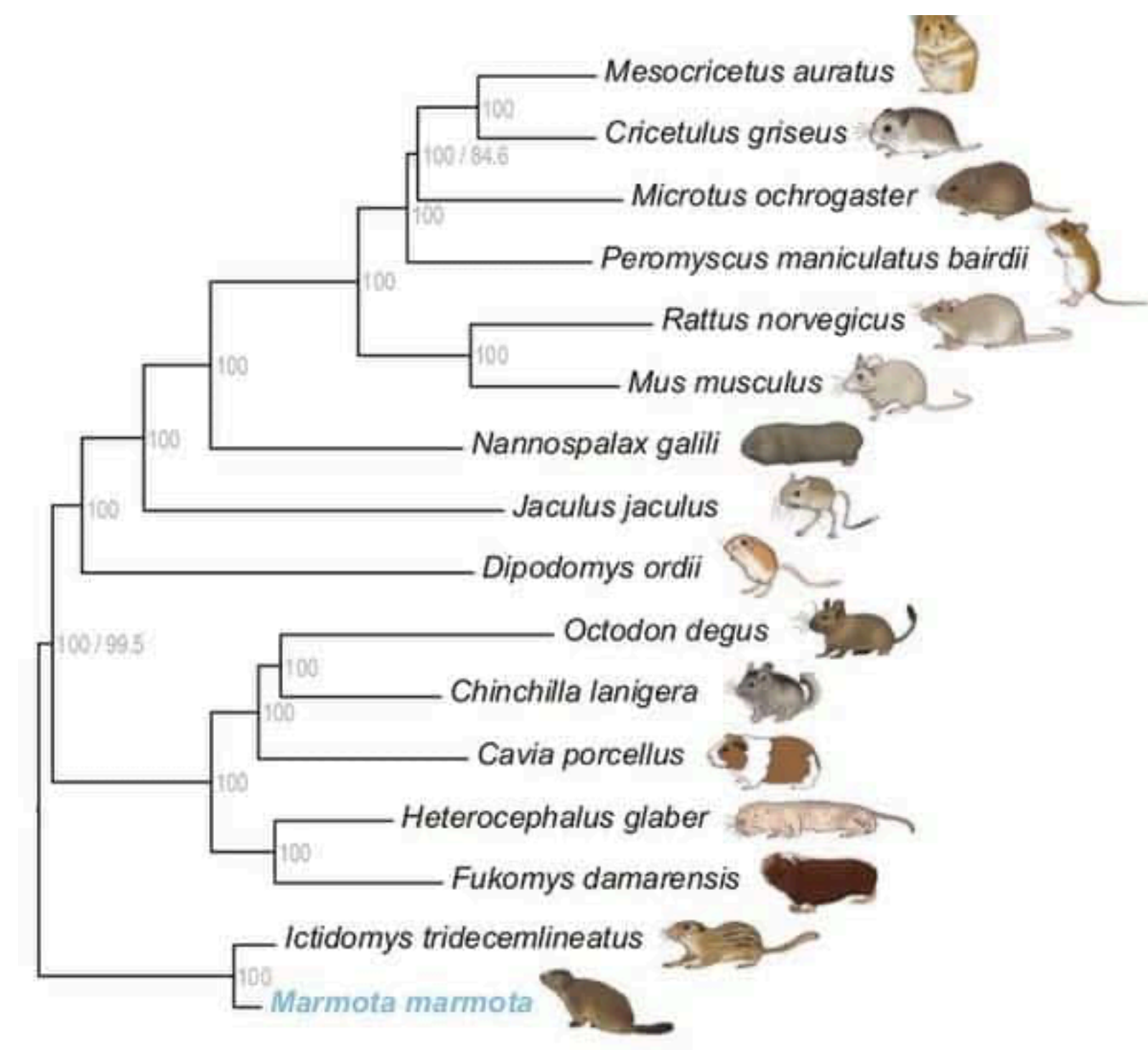
Abstraction: exactly how the class functions is hidden from the programmer

Users of the class don't need to worry about how a class works, just what it does

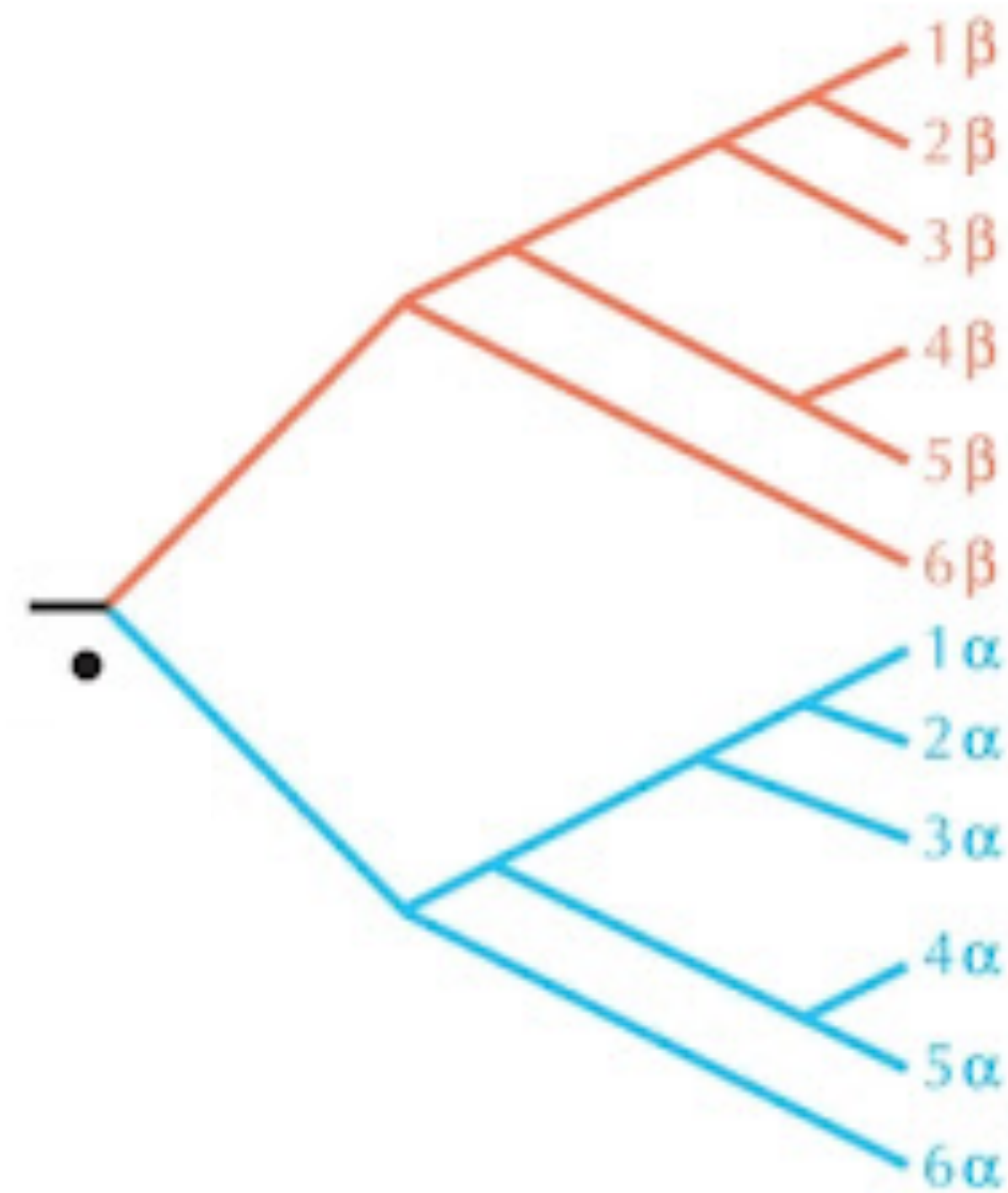
Some internal data is hidden or read-only (**encapsulation**)

ETE4

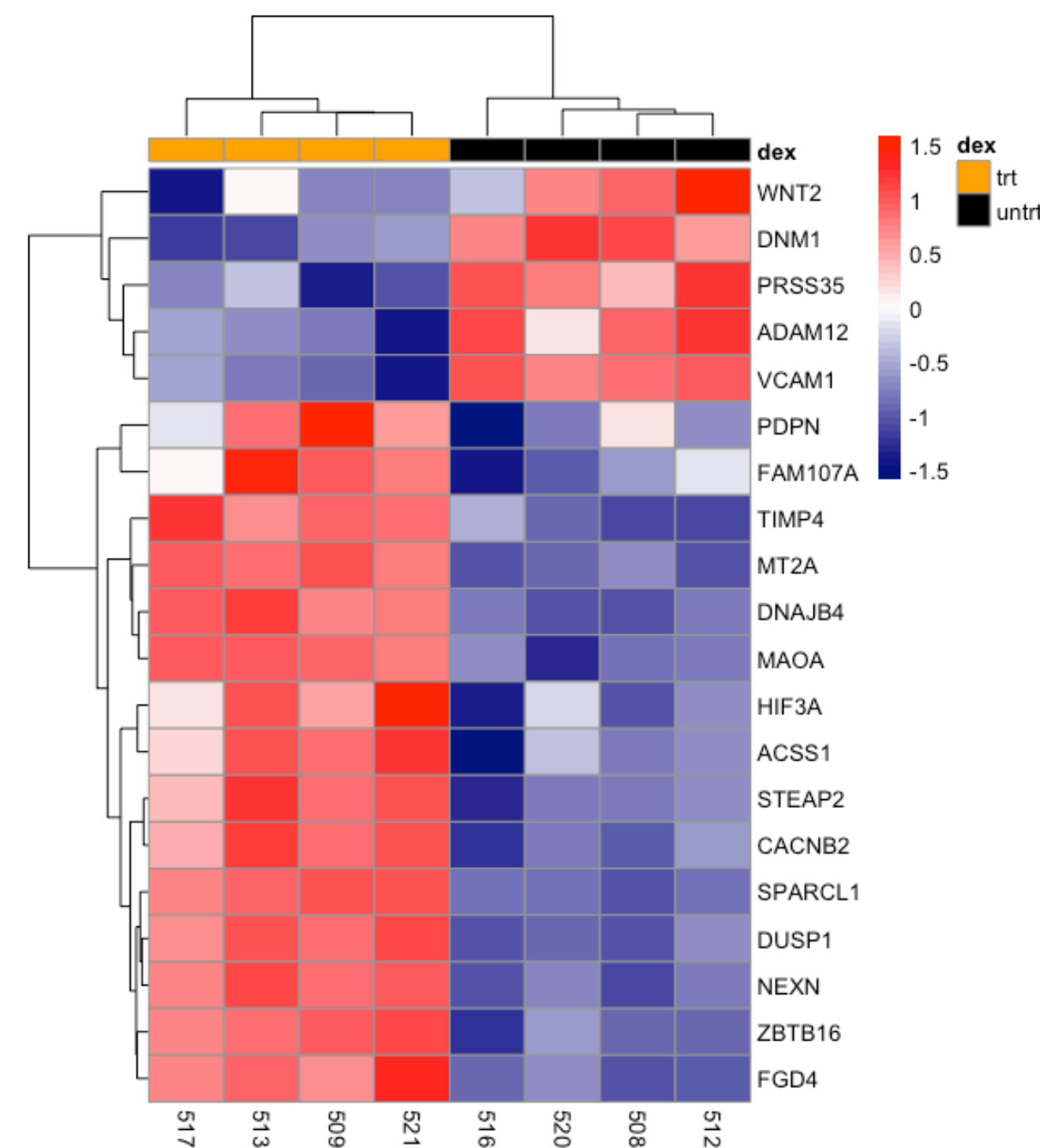
Motivation: working with trees



species tree



gene/protein tree



similarity tree

ETE4

ETE4 is a library for working with tree objects in Python

ETE = environment for tree exploration



ETE4 provides

- Classes for Tree and PhyloTree objects

- Ability to edit, analyze, and compare tree objects

- Ability to link data to nodes (e.g., images, data frames, fasta alignments, etc.)

- Visualization tools for trees and associated data

ETE4 illustrates principles of OOP

Custom classes

Tree class models something in the real world that base Python classes can't model

Inheritance

Tree class is inherited by PhyloTree class (with some additional methods)

Abstraction

Mechanics of how tree topology is encoded is hidden from users

Some of Greg's favorite Python libraries

Polars

Polars is a library for working with data frames in Python



docs.pola.rs

Significant features

Not memory-bound: works with data streams

Very fast: written in Rust; uses all cores / GPUs; workflow optimization; lazy execution

More data types: flexibility and control; supports nested data types

Intuitive interface: rich, expressive set of methods; interacts directly with databases

No automatic index: clear, consistent indexing; eliminates need for many methods

"Come for the speed, stay for the syntax"

Why I like it

Can work with very large datasets on your laptop, bypassing the need for a cluster

Powerful set of methods; clearer, cleaner code

Marimo is a notebook environment for Python



Significant features

Reactive: change code in one cell and affected cells update

Interactive: widgets (sliders, drop-downs, etc.); selectable plots; interactive data frames

Pure Python: text-only files; allows version control with git; can run as scripts

First-class SQL: built-in SQL cells; native support for DuckDB, SQLite, etc.

AI-native editor: built from the ground up to integrate diverse AI tools

Why I like it

Reactivity makes it easier to write and debug code

Encourages reproducibility by simplifying version control

Easy to transition from developing code to using code

UV is a package and project manager for Python

Significant features



docs.astral.sh/uv/

Ultra-fast: written in Rust, *much* faster than predecessors

Integrated: install/update software + manage virtual environments + run scripts

Simple: location-based; fewer commands to manage virtual environments

Reproducibility: uses a .toml file to record versions and dependencies

Space-efficient: uses a global cache to avoid duplicating software

Why I like it

Simpler than working with conda environments

Replaces two different tools (pip and conda)

plotnine is a library for data visualization in Python

Significant features

Logical syntax: based on the grammar of graphics

Familiar: ggplot2 code requires minimal modification

Nice plots: publication-quality output, cleaner than Matplotlib



plotnine.org

Why I like it

No need to learn another plotting syntax; simply apply knowledge of ggplot2

Developed by Duke Professor Mine Cetinkaya-Rundel (also affiliated with Posit)

